

AI-Based Document Screening & Tamper Detection System

Problem Statement ID: 26188

Officer-Facing Border/Airport Security Screening Prototype

SIH Demonstration Project

Technical Architecture & Implementation Guide

Generated: 04-09-2026

Table of Contents

1	Project Overview
2	Technology Stack & Rationale
3	System Architecture
4	Database Design (PostgreSQL)
5	Backend Modules Detailed
5.1	Authentication Module
5.2	Case Management
5.3	OCR Module (PaddleOCR)
5.4	MRZ Module (Passport Only)
5.5	Validation Module
5.6	Tampering Detection (Core Innovation)
5.7	Face Verification Module
5.8	Cross-Document Verification
5.9	Risk Assessment Engine
6	Frontend Pages Detailed
6.1	Page 1: Officer Login
6.2	Page 2: Officer Dashboard
6.3	Page 3: Verification Details
7	End-to-End Data Flow
8	Security & Privacy
9	Deployment Architecture
10	Development Workflow
11	Why This Architecture
12	File Count Summary
13	Running Without Docker

1. Project Overview

This project is an officer-facing AI-assisted border/airport identity and document screening prototype. The system is NOT a passenger self-service system. A trained border/security officer operates the system to upload/scan passenger documents and face captures, while the system performs AI-assisted verification and presents evidence-based results to the officer. The AI provides screening/risk signals; the officer remains the final decision-maker.

Key Constraints

- Prototype for SIH demonstration - not a production immigration system
- No connection to real government immigration/blacklist databases
- Uses synthetic/demo/reference data only
- Modular architecture for future real database integration
- Three main pages: Login, Dashboard, Verification Details

Core Workflow

Officer Login -> Scan/Upload Passenger Documents -> Documents Enter Processing Queue -> OCR Extraction -> Passport MRZ + Checksum -> OCR<->MRZ Cross-Field Verification -> Document Validation -> Cross-Document Verification -> AI Tampering Detection (RGB + Image Forensics + Noise Residual + Tamper Heatmap) -> Face Verification -> Explainable Risk Assessment -> Officer Dashboard -> Final Verification Summary -> Completion

2. Technology Stack & Rationale

Backend Stack

- **Python 3.10+:** Best AI/ML ecosystem (PyTorch, OpenCV, PaddleOCR), async support, type hints
- **FastAPI:** Async native, auto OpenAPI docs, Pydantic validation, high performance, DI
- **SQLAlchemy 2.0 (async):** Mature ORM, async PostgreSQL support, declarative models
- **PostgreSQL 15:** Relational integrity, JSONB for ML results, ACID, production-ready
- **Alembic:** Version-controlled schema migrations, auto-generation
- **JWT + bcrypt:** Stateless auth, scalable, industry standard
- **Celery + Redis:** Background ML processing without blocking API

Frontend Stack

- **TypeScript:** Type safety across API contracts, compile-time error detection
- **React 18:** Component-based, concurrent features, large ecosystem
- **Vite:** Instant HMR, fast builds, ES modules native
- **Tailwind CSS:** Utility-first, consistent design system, small bundle
- **React Router v6:** Declarative routing, nested routes, lazy loading
- **Axios:** Interceptors, request/response transforms, TS support
- **Lucide React:** Clean, consistent, tree-shakeable icons

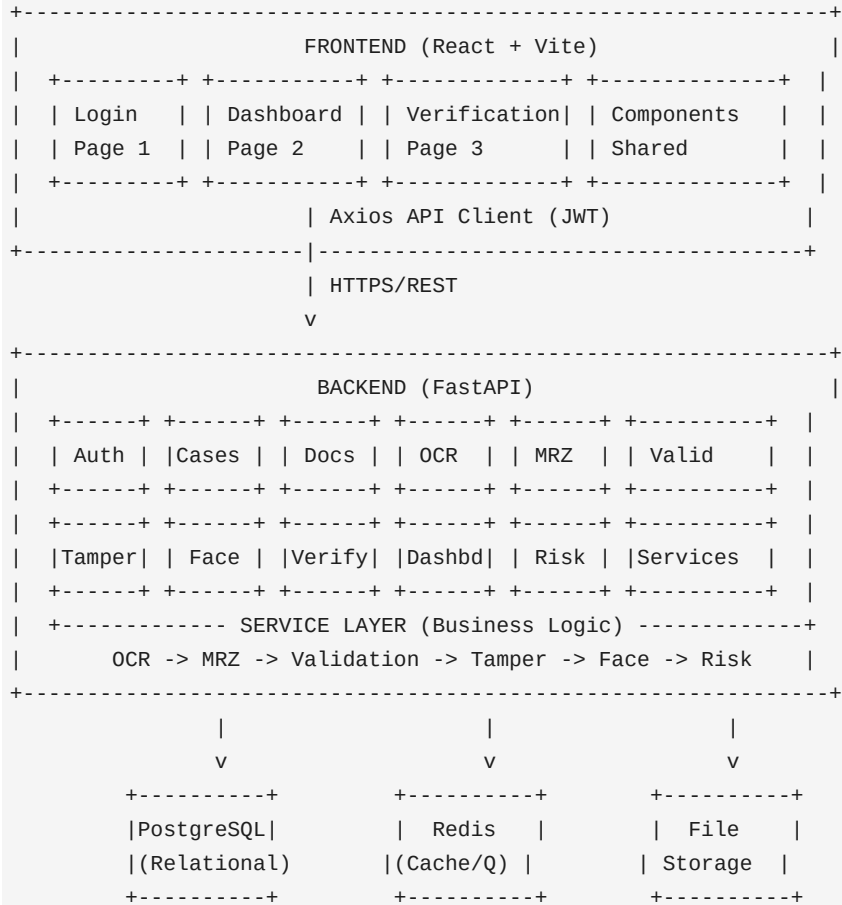
AI/ML Pipeline

- **PaddleOCR:** Multi-language OCR, high accuracy, structured document support
- **Tesseract + Custom Parser:** ICAO TD3 MRZ extraction, checksum validation
- **InsightFace (buffalo_l):** SOTA face recognition, 512-d embeddings, ONNX runtime
- **OpenCV + NumPy + scikit-image:** ELA, noise residual, texture analysis, heatmap generation

3. System Architecture

The system follows a clean client-server architecture with a React frontend communicating via REST APIs to a FastAPI backend. The backend orchestrates an asynchronous ML pipeline using Celery/Redis for long-running document processing tasks.

Architecture Diagram



4. Database Design (PostgreSQL)

Core Tables (12 Tables)

The database uses a fully normalized relational schema with JSONB columns for flexible ML results storage.

Table	Purpose	Key Columns
officers	Authentication & audit trail	id, officer_id, email, hashed_
passengers	Passenger identity (synthetic)	id, passenger_id, full_name, d
verification_cases	Main case record per screening	id, case_id, officer_id, passe
documents	Uploaded files per case	id, case_id, document_type, fi
ocr_results	Extracted text with confidence	id, case_id, document_id, extr
mrz_results	Passport MRZ extraction & chec	id, case_id, document_id, mrz_
validation_results	Field validation + reference D	id, case_id, document_id, fiel
tamper_results	Forensic analysis per document	id, case_id, document_id, tamp
face_results	Face comparison results	id, case_id, document_id, docu
cross_document_result	Cross-document consistency	id, case_id, field_name, docum
risk_assessments	Final explainable risk score	id, case_id, risk_level, risk_
audit_logs	Immutable action trail	id, case_id, officer_id, actio

Key Design Decisions

- **Separate tables per module:** Independent querying, clear ownership, scalable
- **JSONB for ML results:** Flexible schema for varying OCR/ML outputs, queryable
- **File paths not BLOBs:** Database performance, backup size, streaming support
- **Enum types in DB:** Data integrity, self-documenting, indexable
- **Cascade deletes:** Cleanup when case deleted, referential integrity
- **Indexes on case_id, document_id:** Fast lookups for dashboard and verification pages

5. Backend Modules Detailed

5.1 Authentication Module

Location: app/services/auth.py, app/api/v1/auth.py

Features: JWT access tokens (HS256, 8-hour expiry), bcrypt password hashing (cost 12), demo officer auto-creation, protected routes via Depends(get_current_officer).

Endpoints

- **POST /api/v1/auth/login:** Returns access_token
- **POST /api/v1/auth/register:** Create officer (admin use)
- **GET /api/v1/auth/me:** Current officer profile
- **POST /api/v1/auth/init-demo:** Seed demo credentials

5.2 Case Management

Location: app/api/v1/cases.py

Core Flow: Create Case -> Upload Documents -> Start Processing -> Poll Queue -> View Results

Key Endpoints

- **POST /api/v1/cases:** Create new verification case
- **GET /api/v1/cases:** Paginated case list
- **GET /api/v1/cases/{case_id}:** Case details
- **POST /api/v1/cases/{case_id}/documents:** Upload document (multipart)
- **GET /api/v1/cases/{case_id}/documents:** List case documents
- **GET /api/v1/cases/{case_id}/queue:** Processing queue status
- **POST /api/v1/cases/{case_id}/process:** Trigger async pipeline
- **GET /api/v1/cases/history:** Filtered passenger history
- **GET /api/v1/cases/passengers/search:** Passenger search

5.3 OCR Module (PaddleOCR)

Location: app/services/ocr/ocr_service.py

Pipeline: Image -> PaddleOCR -> Text Detection -> Text Recognition -> Field Classification -> Confidence Scoring

Output Structure

```
{
  "extracted_fields": [
    {"field_name": "passport_number", "value": "Z1234567", "confidence": 0.98},
    {"field_name": "full_name", "value": "RAJESH KUMAR SHARMA", "confidence": 0.95}
  ],
  "confidence_scores": {"passport_number": 0.98, "full_name": 0.95},
  "raw_text": "PASSPORT NUMBER: Z1234567
FULL NAME: RAJESH KUMAR SHARMA..."
}
```

Field Classification: Keyword-based mapping for passport_number, full_name, date_of_birth, nationality, gender, date_of_expiry, visa_number, valid_from, valid_until, id_number, permit_type, etc.

5.7 Face Verification Module

Location: app/services/face/face_service.py

Pipeline: Document Image -> Face Detection -> Alignment -> Embedding (512-d) | Live Capture -> Same -> Cosine Similarity -> Match Status

Thresholds (Prototype)

- **>= 0.75:** MATCH
- **0.50 - 0.75:** POSSIBLE_MISMATCH
- **< 0.50:** MISMATCH

Important: Never makes legal determination - only provides evidence for officer review.

5.8 Cross-Document Verification

Location: app/api/v1/cases.py (process_cross_document_verification)

Compared Fields: full_name, date_of_birth, passport_number, nationality, gender

Logic: For each field, collect values from all document OCR results. If all identical -> CONSISTENT. Else -> MISMATCH with details.

Example Output

```
DOB:      Passport=15/08/1985, Visa=15/08/1985, National_ID=15/08/1985 -> CONSISTENT
Passport#: Passport=Z1234567, Visa=Z9999999 -> MISMATCH
```

5.9 Risk Assessment Engine

Location: app/services/risk/risk_engine.py

Weighted Factor Model:

Factor	Weight	Description
Document Validation	15%	All fields valid, reference DB
MRZ Checksum	10%	Passport MRZ checksums pass
OCR->MRZ Consistency	15%	Field-level match percentage
Cross-Document Consistency	15%	Identity fields match across d
Tampering Detection	25%	Max tampering probability acro
Face Verification	15%	1 - similarity_score
Document Expiry	5%	Any expired document

Risk Levels

- **Score <= 0.30:** LOW (Green)
- **0.30 < Score <= 0.60:** MEDIUM (Amber)
- **Score > 0.60:** HIGH (Red)

Explainability: Every factor shows contribution, weight, and human-readable description.

- Officer ID + Password with show/hide toggle
- JWT token storage in localStorage with auto-redirect on valid session
- Demo credentials displayed for testing (OFFICER001 / SecurePass123!)
- Error handling with inline messages
- Visual: Navy background with subtle grid pattern, royal blue primary button

+-----+					
VERIFICATION TIMELINE (visual progress with icons)					
+-----+					
DOCUMENTS MRZ DETAILS VALIDATION CROSS-DOC TAMPER FACE					
[Expandable sections with tables, images, heatmaps]					
+-----+					
FINAL RISK ASSESSMENT (Prominent color-coded card)					
Contributing factors with weights & descriptions					
+-----+					
FINAL SUMMARY (Condensed decision-ready view)					
DOCUMENT STATUS MRZ CROSS-DOC TAMPERING FACE					
OVERALL RISK: HIGH [COMPLETE VERIFICATION]					
+-----+					

Key UI Components

- Timeline - Visual progress with icons (CHECK/LOADING/WAITING/WARNING/FAIL)
- MRZ Table - Field | OCR Value | MRZ Value | Result (CHECK/FAIL)
- Tamper Panel - 3-column: RGB Image | Heatmap | Noise Residual
- Face Comparison - Side-by-side with circular similarity % indicator
- Risk Panel - Color-coded, factor breakdown with weights
- Final Summary - Condensed decision-ready view for officer

7. End-to-End Data Flow

- **1. OFFICER LOGIN:** POST /auth/login -> JWT -> stored in localStorage
- **2. CREATE CASE:** POST /cases {case_id, passenger_id} -> VerificationCase record
- **3. UPLOAD DOCUMENTS:** POST /cases/{id}/documents (multipart) -> Document records + files saved
- **4. START PROCESSING:** POST /cases/{id}/process -> Background task triggered
- **5. BACKGROUND PIPELINE:** Async processing per document (see below)
- **6. POLLING:** GET /cases/{id}/queue every 2s -> Updates UI progress bars
- **7. VIEW RESULTS:** GET /verification/{id}/detail -> Complete VerificationDetail response
- **8. COMPLETE:** POST /verification/{id}/complete -> Audit log entry

Background Pipeline (Per Document)

- OCR Extraction (PaddleOCR) -> ocr_results
- IF Passport: MRZ Pipeline -> mrz_results (Detection, Parse, Checksum, OCR<->MRZ Comparison)
- Document Validation -> validation_results (Required fields, Format, Expiry, Reference DB)
- Tampering Detection -> tamper_results (RGB, Noise, Forensics, Heatmap)
- IF Passport: Face Verification -> face_results (Detect, Embed, Compare, Match Status)

After all documents: Cross-Document Verification -> Risk Assessment (aggregate weighted factors, apply thresholds, generate explanations)

8. Security & Privacy

- **Authentication:** JWT with 8hr expiry, bcrypt passwords
- **Authorization:** Officer-scoped data access via middleware
- **CORS:** Restricted to frontend origin
- **File Upload:** Type validation, 10MB limit, secure filenames
- **Data Storage:** No real PII - synthetic demo data only
- **Audit Trail:** Every action logged with officer, timestamp, details
- **No Real DB Connections:** Mock reference database clearly labeled
- **Prototype Disclaimers:** UI states 'DEMO/REFERENCE DATA' throughout

9. Deployment Architecture

Docker Compose Services

Service	Port	Purpose
postgres	5432	Primary database
redis	6379	Cache & Celery broker
backend	8000	FastAPI (uvicorn + reload)
frontend	5173	Vite dev server (HMR)

Production Considerations

- **File Storage:** S3/MinIO with presigned URLs

- **Background Tasks:** Celery workers (separate containers)
- **SSL/TLS:** Nginx reverse proxy + Let's Encrypt
- **Monitoring:** Prometheus + Grafana
- **Logging:** Structured JSON logs -> ELK/Loki
- **Secrets:** Vault or Docker secrets
- **Scaling:** K8s deployment with HPA

10. Development Workflow

Adding a New Feature

- **Step 1:** Check PROJECT_RULES.md - Verify not locked
- **Step 2:** Backend: Add model -> schema -> service -> router -> migrate
- **Step 3:** Frontend: Add types -> API call -> component -> page integration
- **Step 4:** Test: Run both servers, verify flow
- **Step 5:** Document: Update README if new endpoints

Code Quality Commands

```
# Backend
cd backend && ruff check . && pytest

# Frontend
cd frontend && npm run lint && npm run build
```

11. Why This Architecture

- **Officer-facing (not passenger):** Auth, dashboard, case-centric workflow
- **Explainable AI:** Risk engine shows every factor with weight
- **Modular ML pipeline:** Each module independent service, swappable
- **Real-time progress:** Polling queue with status updates
- **Professional UI:** Tailwind design system, government aesthetic
- **SIH Demo Ready:** 5 pre-seeded cases covering all scenarios
- **Extensible:** Clean interfaces for real DB integration
- **Audit Compliant:** Immutable logs for every verification

12. File Count Summary

Total: ~45 source files

Backend (23 files)

- app/main.py + config + database
- app/models/models.py (12 tables)
- app/schemas/schemas.py (40+ Pydantic models)
- app/api/v1/ (11 routers)
- app/services/ (6 ML modules)
- alembic/ (migrations)
- scripts/seed_demo_data.py
- requirements.txt, Dockerfile, .env.example

Frontend (18 files)

- src/pages/ (3 main pages)
- src/components/common/ (Layout, UI kit)
- src/services/api.ts

- src/hooks/useAuth.tsx
- src/types/index.ts (all interfaces)
- src/utls/helpers.ts
- package.json, tsconfig, vite.config, tailwind.config
- Dockerfile, .env

Root (3 files)

- docker-compose.yml
- PROJECT_RULES.md
- README.md

13. Running Without Docker

Prerequisites

Tool	Download
Python 3.10+	https://python.org/downloads
Node.js 18+	https://nodejs.org
PostgreSQL 15	https://www.postgresql.org/dow
Redis 7	https://redis.io/download (Win

Backend Setup

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
# Edit .env with PostgreSQL credentials
alembic upgrade head
python scripts/seed_demo_data.py
uvicorn app.main:app --reload --port 8000
```

Frontend Setup

```
cd frontend
npm install
echo VITE_API_URL=http://localhost:8000/api/v1 > .env
npm run dev
```

Verify

Service	URL	Test
Backend API	http://localhost:8000/docs	Swagger UI loads
Frontend	http://localhost:5173	Login page appears
Login	OFFICER001 / SecurePass123!	Redirects to dashboard

End of Document

AI-Based Document Screening & Tamper Detection System

Problem Statement ID: 26188

This document serves as the complete technical reference for the project.

All code is located in C:\Users\Abhiraj Mangire\Documents\AI-Border-Screening